NTNU

PRE-PROJECT REPORT

Creating a robotic arm for a drone battery swapping station

AIS4104 Robotics and Intelligent Systems with Project

Master in Mechatronics and Automation
Spring 2025

Faculty of Information Technology and Electrical Engineering (IE)
Norwegian University of Science and Technology (NTNU)

Jon Urcelay

January 31, 2025

Contents

List of Figures

List of Tables

Acronyms

DBS Drone Battery Swap.

ROS Robot Operating System.

DoF Degrees of Freedom.

UR Universal Robots.

CV Computer Vision.

CNN Convolutional Neural Networks.

R-CNN Region-Based Convolutional Neural Networks.

YOLO You Only Look Once.

SSD Single Shot Multibox Detector.

API Application Programming Interface.

DDS Data Distribution Service.

OS Operating System.

1 Introduction

An automatic DBS station is a mechanism that changes the battery of a drone, removing the low charge battery from the drone and replacing it with a fully charged one. A 3D representation of the station can be seen in the next illustration:

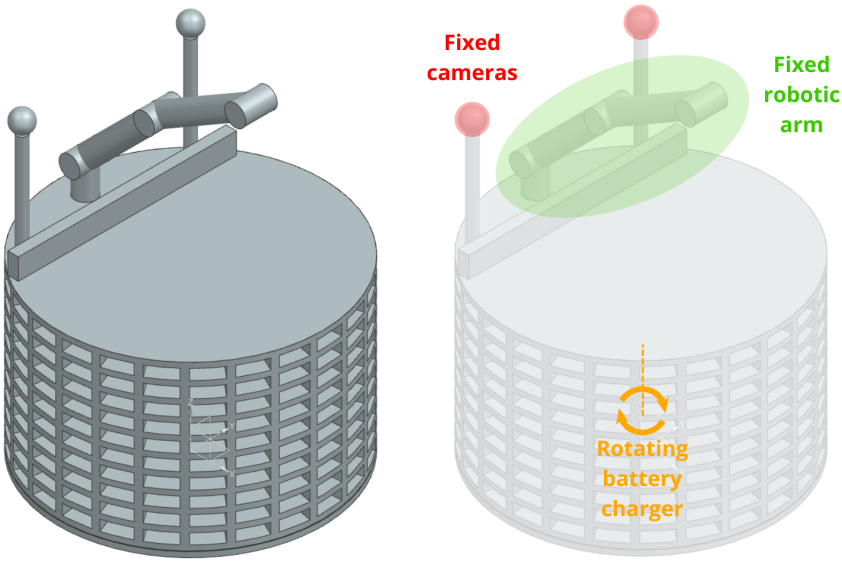

Figure 1: 3D representation of the DBS station

2 Problem statement

For this project, the objective is to develop a simplified prototype of the station, which consists of a **6-DoF UR3e robotic arm with computer vision**. The simplifications made from the original concepts are the next (see Figure {2} while reading):

- **Batteries.** Instead of using real batteries, 3D printed parts will be used.
- **Charging area.** Instead of the original rotating concept, a simple shelf-type structure will be placed next to the robotic arm, its location being known and constant. This structure will hold at least two batteries.
- **Drone.** Instead of a drone, a simplified 3D printed structure will be used, which will have room for one battery.
- **Drone battery direction.** It will be assumed that the drone will always land facing the compartment containing the battery towards the UR3e robot. However, the landing won't be the same every single time.
- **Horizontal battery orientation.** It will be assumed that the battery that comes in the drone and the batteries that are being charged are oriented in the horizontal direction.

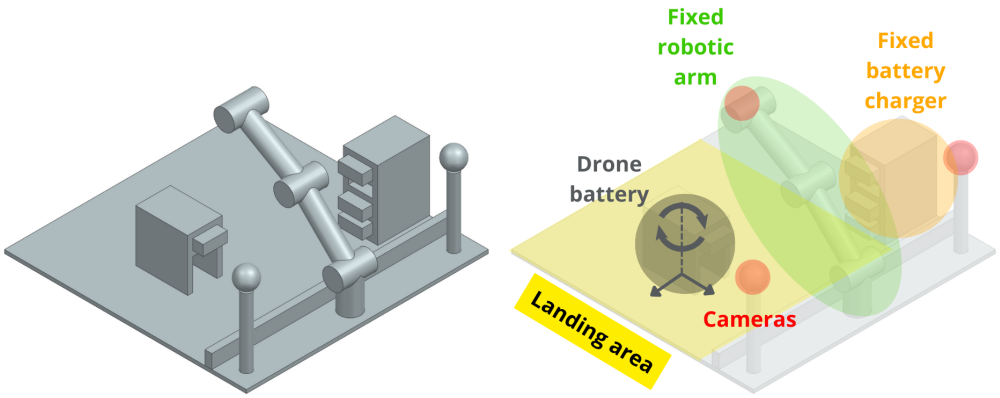

Figure 2: 3D representation of the simplified DBS station

In this way, after these simplifications have been performed, the main focus of the project can be put on computer vision and motion planning of the robotic arm.

As represented in Figure {2}, the simplified DBS station will have the following groups of components:

- **Landing area.** Visible area for the cameras, where the drone battery will be placed.
- **Drone battery.** Can be translated in the two directions of the landing area and rotated in the vertical axis.
- **Fixed battery charger.** One of the compartments from the battery charger will be empty, prepared for the battery that is coming from the drone. The rest of the compartments will be filled with batteries, ready to be picked up and placed in the drone.
- **Cameras.**
  - **Fixed cameras.** When the drone lands in the landing area, the fixed cameras will detect it's position, so that the robotic arm can go there.
  - **End-effector camera.** It will detect the empty compartment of the fixed battery charger and a new battery to pick up.
- **Fixed robotic arm.** A 6 DoF UR3e robotic arm (Figure {3}) will be used for doing the battery swap. The end-effector used with the UR3e robotic arm will be the 2FG7 OnRobot parallel gripper (Figure {3}), an electric gripper with grip detection.

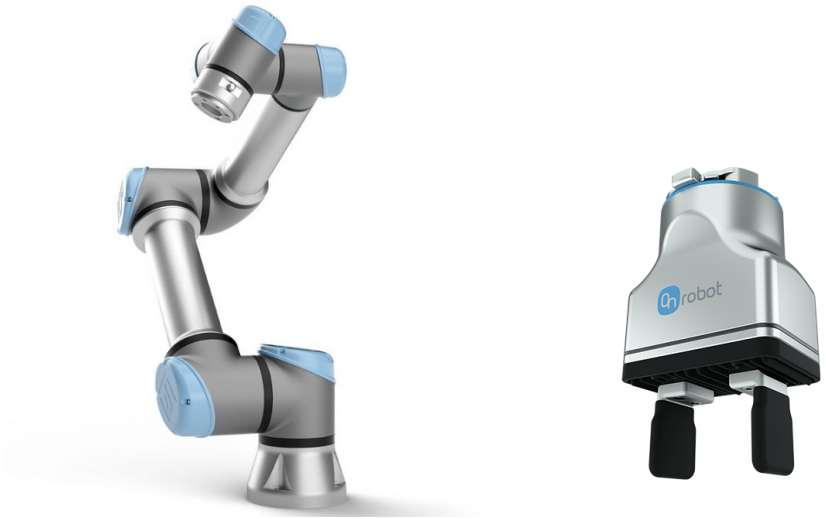

Figure 3: UR3e robotic arm (left) & 2FG7 OnRobot parallel gripper (right)

2.1 Task order for the battery swap

Task no.	Camera	Computer vision task	UR3e move task
1	Fixed camera	Capture the position of the drone in the landing area	Move to a position next to the drone
2	End-effector camera	Capture the position of the battery in the drone	Take the battery out of the drone
3	-	-	Move to the fixed battery charge station
4	End-effector camera	Capture the position of an empty compartment and of a new battery to pick up	Insert battery in the empty compartment
5	-	-	Take the new battery out of its compartment
6	Fixed camera	Capture the position of the drone in the landing area	Move to a position next to the drone
7	End-effector camera	Capture the position of the empty compartment in the drone	Insert battery in the empty compartment of the drone

Table 1: Task order for the battery swap

The system begins with fixed cameras to capture the landing area and passing the coordinate data of the drone to the UR3e controller. The robotic arm moves to this position and orientation.

Then the end-effector cameras capture the image of the battery, passing the coordinate data to the controller. The robotic arm goes to the specified position following a perpendicular trajectory relative to the direction of the battery, closes the gripper and takes out the battery from the drone.

After that, the robotic arm moves to the fixed battery charge station, where the camera in the end-effector captures an empty compartment and a new battery to pick up, passing the coordinate data to the controller. The UR3e inserts the battery in the empty compartment, following a perpendicular trajectory, opens the gripper and moves back. Next, it moves to the position of the new battery, following a perpendicular trajectory to that point, closes the gripper and takes out the new battery.

Subsequently, the fixed cameras capture the landing area and pass the coordinate data of the empty drone to the controller. The robotic arm moves to this position and orientation.

Finally, the end-effector's camera captures the empty compartment, passing the coordinate data for the last time to the controller. The robotic arm moves to the given point following a

perpendicular trajectory, introducing the battery in the drone's compartment, opens the grip and goes back to the home position. In this way, the battery swap cycle is completed.

3 Goals

3.1 Computer vision

- Detect the position and orientation of the drone battery.
- Detect the empty compartment on the fixed battery charger.
- Detect the new battery to be taken off the fixed battery charger.
- Detect the position and orientation of the empty compartment in the drone battery.

3.2 Motion planning of UR3e

- Generate the trajectories given the desired points to reach.

3.3 System integration

- Include closing and opening for end end-effector
- Use ROS to coordinate all the modules in computer vision, motion planning and end-effector manipulation.

4 Methods to achieve the goals

4.1 Computer vision

The design begins with taking an image and ends with the output in the form of a data package containing position data and the orientation angle of the object. For this, two methods are proposed, ArUco markers and an object detection algorithm called R-CNN with InceptionV2 .

2D cameras will be used for taking the pictures, both from fixed positions in the station and from the end-effector of the UR3e. A webcam that can be used for this function is the Logitech C270 (see Figure {4}).

Figure 4: Logitech C270 webcam

4.1.1 ArUco markers

ArUco markers are square-shaped patterns with a unique binary code inside, which makes them easily identifiable by computer vision algorithms. They are used for object detection and pose estimation.

If using ArUco Markers(Figure {5}), it would be necessary to print and attach them to each of the batteries, as well as to the drone battery compartment, to some fixed positions in the landing area and to the fixed battery charger.

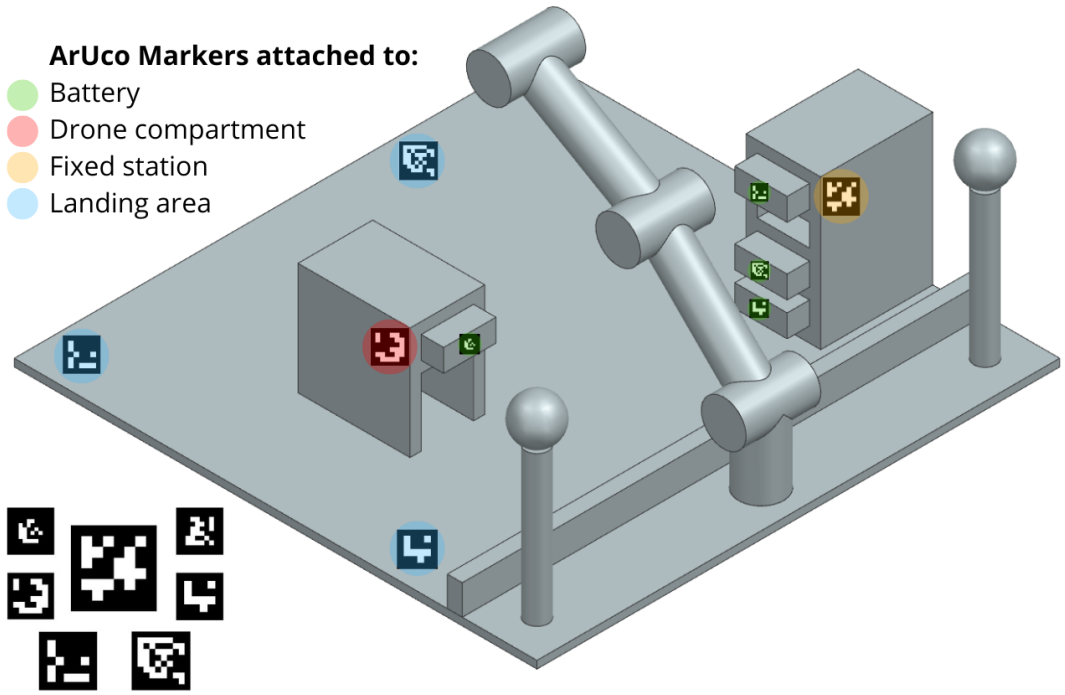

Figure 5: ArUco markers

For using ArUco markers, the following tools are needed:

- **OpenCV.** For detecting ArUco markers, estimating their pose and orientation, and calibrating the camera.
- **ArUco module in OpenCV.** Specifically designed for generating, detecting, and estimating the pose of ArUco markers.
- **Stereo Vision.** Needed for computing depth from two C270 cameras.

4.1.2 Object detection algorithm: R-CNN with InceptionV2

An object detection algorithm in CV is a method used to identify and locate objects within an image or video, but in this case without using any kind of markers, just the object itself. These algorithms analyze visual data to detect specific patterns, shapes or features that correspond to the objects of interest. Some common types of object detection algorithms in computer vision are CNNs, YOLO, SSD and R-CNN.

If using an object detection algorithm to detect batteries and compartments, the 3D model of the batteries and compartments would need to have a recognizable design, to make them easier to be detected by the algorithm.

For this project, the use of Faster R-CNN with InceptionV2 is proposed. This provides high accurate detection capabilities, which is needed for detecting the position of batteries and compartments. Faster-R-CNN is a single, unified network for object detection. It uses region proposal network (RPN) module that serves as the attention mechanism. It tells the unified network where to look. On the other hand, Inception is composed of efficient inception module with 22 layers and no fully connected layers. The main advantage of this model is the improved utilization of the computing resources inside the network. It's computationally intensive but produces more accurate results in object detection.

The object detection algorithm will be trained using a labeled dataset of images. These images will be taken from the 3D printed parts. In addition, the trained model will be tested on a separate dataset of pictures, to evaluate its performance and make necessary adjustments.

For using Faster R-CNN with InceptionV2, the next packages, libraries and tools are needed:

- **OpenCV.** For image pre-processing, camera calibration, and feature extraction.
- **TensorFlow.** To develop and train the Faster R-CNN model.
- **TensorFlow Object Detection API.** Provides pre-trained models and tools for training and deploying object detection models.
- **Stereo Vision.** Used for depth estimation without relying on markers.

4.2 Motion planning of UR3e

Tools required for UR3e robotic arm control:

- **Universal Robots ROS Driver.** Official ROS2 driver for controlling UR3e.
- **PolyScope & URCap.** Graphical programming interface of UR. When the robot starts, the user has the choice to run an existing program, create or modify a program, or configure the installation / environment settings of the robot. URCap is a Java-based plugin, that integrates in to PolyScope the graphical programming interface of UR. URCaps can be used to extend the functionality of PolyScope, for example by creating new friendly programming screens, so that the user can configure additional hardware like cameras and grippers. These two are necessary but primarily for setup, not real-time control.
- **MoveIt2.** A motion planning framework that integrates with ROS2 to control the UR3e robotic arm. It helps with inverse kinematics, collision avoidance, and motion planning.

For testing motion sequences before real-world execution, **Ignition Gazebo** will be used. This is a simulation environment for testing the robotic arm and vision system in a virtual environment using ROS2.

Tools for coordinate transformation and object localization:

- **Transformation library tf2-ros.** For managing coordinate frames and transformations between different parts of the system (camera, markers and robot base).

- **cv2.solvePnP().** Estimates 3D object position and orientation from ArUco markers or 2D detection points. Necessary for accurate 3D localization.
- **Eigen.** For using the necessary transformation matrices.

4.3 System integration

To connect the different part of the hardware.

- **ROS2 Jazzy.** ROS is a robotics middleware for real-time communication, control, and simulations. ROS2 Jazzy is the latest version of ROS, which supports better real-time control and improved DDS communication. **Ubuntu 24.04** is the officially required OS for running ROS2 Jazzy and related packages.
- **WSL 2.** Windows Subsystem for Linux 2, which allows to run Linux distributions on Windows, so that ROS2 development can be done inside Windows without dual-booting. It provides faster file system performance and GPU passthrough for AI workloads. It allows using Windows software (VS Code, TensorFlow, PyTorch, etc.) alongside Linux tools.
  Issues with WSL2:
  - Limited access to USB devices.
  - Runs inside Windows, which does not support real-time execution for robotics.
  - No native support for Gazebo physics engine acceleration.
- **Virtual Machine VMware Workstation Player.** Since WSL 2 has limitations (USB access, real-time performance, Gazebo simulation), using a VM is a better alternative for running ROS2 Jazzy with UR3e, TensorFlow and Gazebo.

5 Materials and budget

Hardware:

Object	On campus	Link	Price [NOK]
UR3e Robotic Arm	Yes	UR3e	0
UR3e Teach pendant (PolyScope)	Yes	UR3e	0
UR3e Controlbox	Yes	UR3e	0
2FG7 OnRobot parallel gripper	Yes	2FG7	0
OnRobot Eyes	Not sure	OnRobot Eyes	0
OnRobot Eye Box	Not sure	OnRobot Eyes	0
Logitech C270 webcam x 2	No	Logitech C270	660
3D printers and PLA	Yes		0
Computer	Yes		0
ArUco Markers (printed)	Yes	ArUco Generator	0
Total			660

Table 2: Hardware

Software:

Tool	Available	Link	Price [NOK]
OpenCV	Yes	OpenCV Github	0
TensorFlow	Yes	Tensorflow Github	0
UR ROS Driver	Yes	UR ROS2 Driver Github	0
PolyScope & URCap	Yes	PolyScope & URCap Github	0
MoveIt2	Yes	MoveIt2 Github	0
Eigen	Yes	Eigen	0
ROS2 Jazzy	Yes	ROS2 Jazzy Installation	0
Ubuntu 24.04	Yes	Ubuntu 24.04 Installation	0
WSL 2	Yes	WSL2 Installation	0
VMware Workstation Player	Yes	VMware Installation	0
Ignition Gazebo	Yes	Ignition Gazebo Installation	0
Total			0

Table 3: Software

6 Preliminary progress plan

6.1 Initial setup for ArUco markers

- Create the next parts using 3D printers:
  - Batteries.
  - Fixed battery compartments.
  - Movable battery compartment.
  - Structure to hold the cameras in a fixed position.
  - Structure to hold the camera in the end-effector.
- Place and connect the cameras in a fixed position on the platform and on the end-effector, using the 3D printed structures.
- Place the 3D printed fixed battery compartments in a fixed position in the platform.
- Print ArUco markers and place them in their places.
- Set up OpenCV and make sure that they detect the ArUco markers.
- Set up ROS2 and makes ure that it moves the UR3e robotic arm.

6.2 Motion planning with ArUco markers

6.3 Testing and documentation

6.4 Initial setup for R-CNN

- Modify and print the next parts using 3D printers, removing the location for ArUco markers:
  - Batteries.
  - Fixed battery compartments.
  - Movable battery compartment.
- Place the 3D printed fixed battery compartments in a fixed position in the platform.
- Set up OpenCV for R-CNN with Inception2.
- Set up ROS2 and makes ure that it moves the UR3e robotic arm.

6.5 Training the model

6.6 Motion planning with R-CNN with Inception2

6.7 Testing and documentation

7 Relation with chapters from the book

In this chapter the proposed chapters from the book are mentioned, pointing out if this specific project has a direct relation with each of them or not.

Chapter 7. Kinematics of closed chains.
There is no relation with this chapter, as the 6 DoF robot is not a closed chain.

Chapter 8. Dynamics of open chains.
There is no relation with this chapter, as no forces or accelerations are considered.

Chapter 10. Motion planning.
There is a direct relation with this chapter, as the robotic arm will need to calculate the trajectories to complete the battery swap.

Chapter 11. Robot control (motion control, force control, etc).
There is no relation with this chapter, as the robot control is made in the controller of the UR3e. The calculated trajectories will be the input to the robot controller, but the input to the actuators itself is made by the controller and are not covered in this project

Chapter 12. Grasping and manipulation.
There is no relation with this chapter, as neither of the end effector contact kinematics, forces and/or dynamic will be considered in this project.

Chapter 13. Wheeled mobile robots.
There is no relation with this chapter, as there are no mobile robots in the project.

In conclusion, **Chapter 10** is the one that will be used in this project.

8 Course synergies

IE509118 Experts in Teams.
In this course a prototype for the battery adapter for the drone will be made. At the same time, business models possibilities will be explored.

9 Research and references for methods

1 - Image Processing based UR5E Manipulator Robot Control in Pick and Place Application for Random Position and Orientation of Object.

2 - An ArUco-based Grasping Point Detection System

3 - Object Detection Using Convolutional Neural Networks.

4 - Object detection 3-D coordinate extraction and pose estimation for an autonomous medical robot

10 Comments from revision with Aleksander

- Start simple, up and running, and then make it complex
- Intel realsense d515 Camera available